Worksheet -3

Name: BINA THAPA

Module Title: Programming in C++

Student id- 23085129

Task 1 : 30 marks

1. Create a Time class to store hours and minutes. Implement:
    1. Overload the + operator to add two Time objects
    2. Overload the > operator to compare two Time objects
    3. Handle invalid time (>24 hours or >60 minutes) by throwing a custom exception

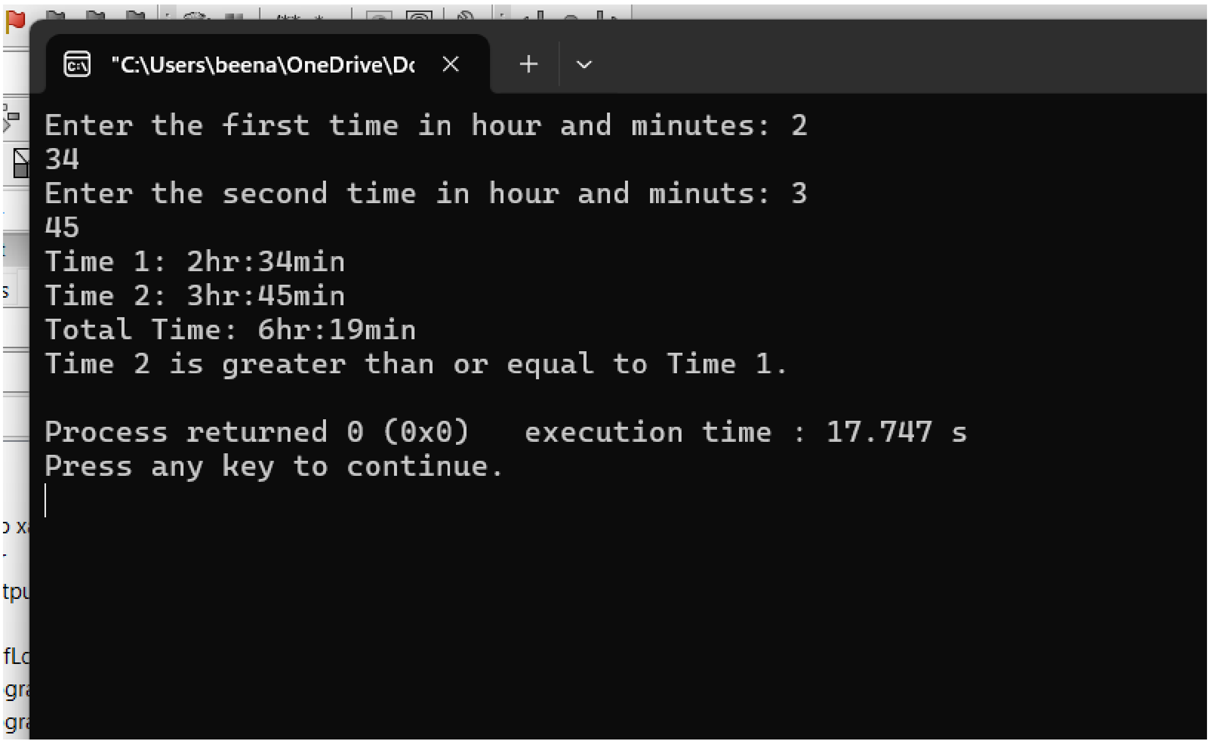
    - Answer:

```
#include <iostream>
using namespace std;
class Time
{
private:
   int hours;
   int minutes;
   string error;
public:
    Time()
    {
        hours=0;
        minutes=0;
    }

     Time(int a,int b)

   {
       if (a > 24 || b > 60 || a < 0 || b < 0)
       {
           throw string("Invalid Input!!");
       }
       hours=a;
       minutes=b;

   }

    Time operator+(Time T)
    {
        Time temp;
        temp.minutes=minutes+T.minutes;
        int tt=temp.minutes/60;
        temp.minutes=temp.minutes%60;
        temp.hours=hours+T.hours+tt;
```

```
        return temp;
    }
    bool operator>(Time T)
    {
       int total1 = hours * 60 + minutes;
       int total2 = T.hours * 60 + T.minutes;
       return total1 > total2;
    }
    void display()
    {
       cout<<hours<<"hr"<<":"<<minutes<<"min"<<endl;
    }
};
int main()
{
    try {
       int h1, m1, h2, m2;

       cout << "Enter the first time in hour and minutes: ";
       cin >> h1 >> m1;
       Time t1(h1, m1);

       cout << "Enter the second time in hour and minuts: ";
       cin >> h2 >> m2;
       Time t2(h2, m2);

       cout << "Time 1: ";
       t1.display();
       cout << "Time 2: ";
       t2.display();


       Time total = t1 + t2;
       cout << "Total Time: ";
       total.display();

       if (t1 > t2)
          cout << "Time 1 is greater than Time 2." << endl;
       else
          cout << "Time 2 is greater than or equal to Time 1." << endl;

    }
    catch (string error)
```

```
    {
        cout << error << endl;
    }

    return 0;
}
```

- Output



Task 2: 70 marks

1. Create a base class Vehicle and two derived classes Car and Bike:
    1. Vehicle has registration number and color

2. Car adds number of seats
3. Bike adds engine capacity
4. Each class should have its own method of writing its details to a file
5. Include proper inheritance and method overriding

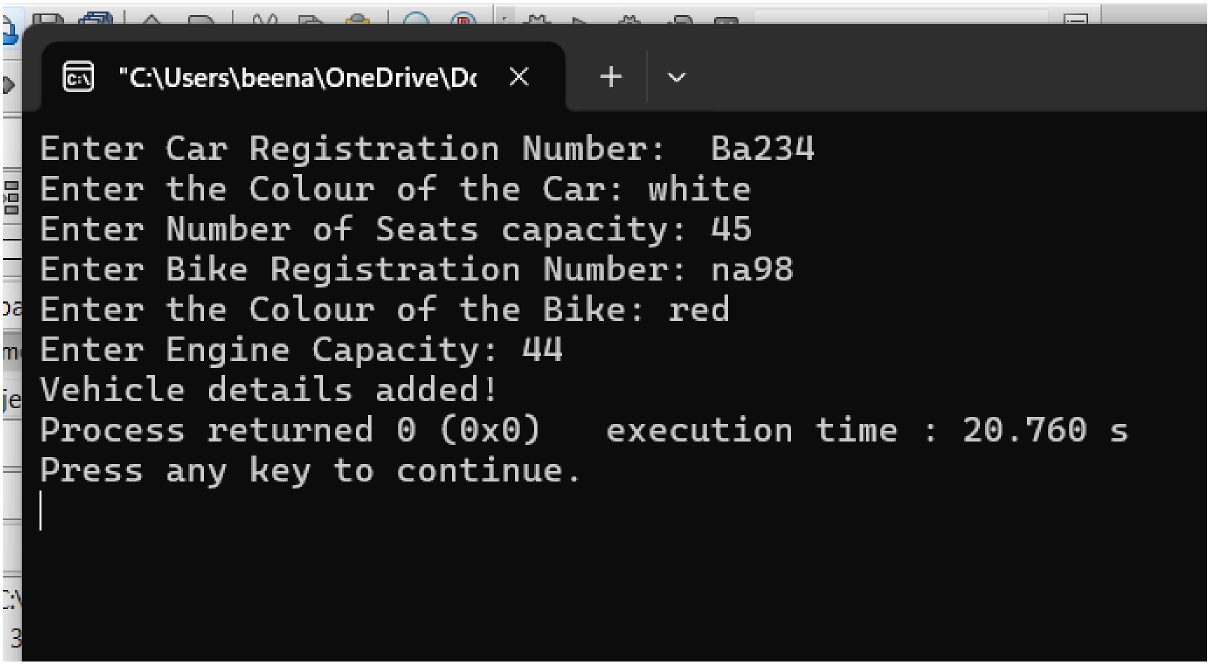
➢ Answer:

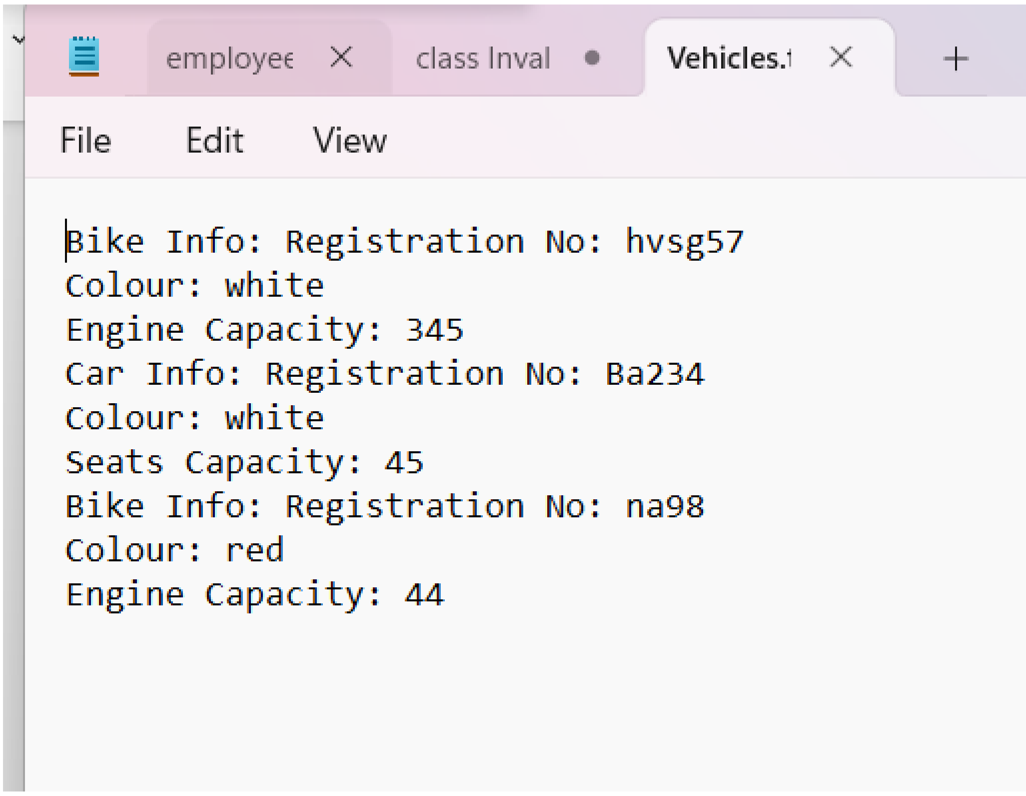
```
#include <iostream>
#include <fstream>
using namespace std;
class Vehicle
{
protected:
   string RegistrationNumber;
   string Colour;
public:
   Vehicle()
   {
      RegistrationNumber="";
      Colour="";
   }
   Vehicle(string a,string b)
   {
      RegistrationNumber=a;
      Colour=b;
   }
   virtual void write()
   {
      ofstream out("Vehicles.txt", ios::app);
      out << "Registration No: " << RegistrationNumber << endl;
      out<< "Colour: " << Colour << endl;
      out.close();

   }
};
class Car:public Vehicle
{
private:
   int seats;
public:
   void CarDetails(string a,string b,int c)
   {
      RegistrationNumber=a;
```

```
        Colour=b;
        seats=c;

    }
    void write()override
    {
        ofstream file("Vehicles.txt", ios::app);
        file << "Car Info: ";
        file << "Registration No: " << RegistrationNumber << endl;
        file << "Colour: " << Colour << endl;
        file << "Seats Capacity: " << seats << endl;
        file.close();
    }

};
class Bike:public Vehicle
{
private:
    int engine;
public:
    void BikeDetails(string a,string b,int c1)
    {
        RegistrationNumber=a;
        Colour=b;
        engine=c1;

    }
    void write()override
    {
        ofstream file("Vehicles.txt", ios::app);
        file << "Bike Info: ";
        file << "Registration No: " << RegistrationNumber << endl;
        file << "Colour: " << Colour << endl;
        file << "Engine Capacity: " << engine << endl;
        file.close();
    }

};
int main()
{
    Car c1;
    Bike b1;
```

```cpp
    string reg, colour;
    int seats, engine;


    cout << "Enter Car Registration Number: ";
    cin >> reg;
    cout << "Enter the Colour of the Car: ";
    cin >> colour;
    cout << "Enter Number of Seats capacity: ";
    cin >> seats;
    c1.CarDetails(reg, colour, seats);


    cout << "Enter Bike Registration Number: ";
    cin >> reg;
    cout << "Enter the Colour of the Bike: ";
    cin >> colour;
    cout << "Enter Engine Capacity: ";
    cin >> engine;
    b1.BikeDetails(reg, colour, engine);


    c1.write();
    b1.write();

    cout << "Vehicle details added!";

    return 0;
}
```

- Output:





Create a program that:

6. Reads student records (roll, name, marks) from a text file
7. Throws an exception if marks are not between 0 and 100

8. Allows adding new records with proper validation
9. Saves modified records back to file

   - Answer:

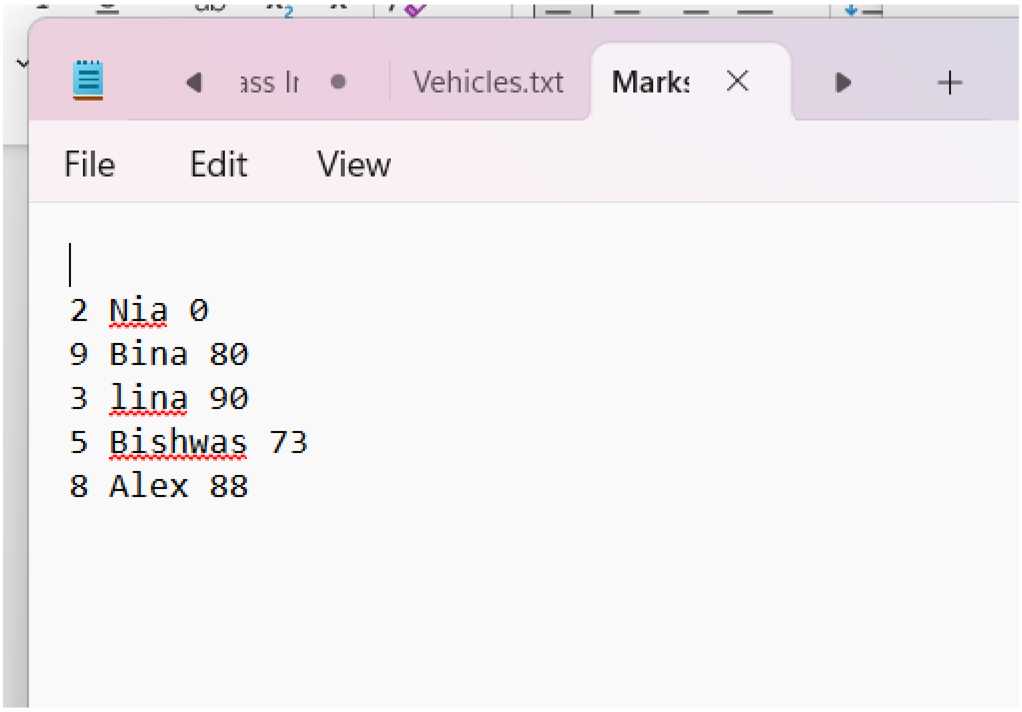
```
#include <iostream>
#include <fstream>
using namespace std;

class Students
{
    int Rollno;
    string Name;
    float Marks;

public:
    Students()
    {
        Rollno = 0;
        Name = "";
        Marks = 0;
    }

    Students(int r, string n, float m)
    {
        if (m < 0 || m > 100)
        {
            throw string("Invalid input!");
        }
        Rollno = r;
        Name = n;
        Marks = m;
    }

    void getinput()
    {
        cout << "Enter Student Details: "<<endl;
        cout << "Student Roll No: "<<endl;
        cin >> Rollno;
        cout << "Student Name: "<<endl;
        cin >> Name;
        cout << "Marks obtained: "<<endl;
        cin >> Marks;
```

```
        if (Marks < 0 || Marks > 100) {
            throw string(" Invalid input!");
        }
    }

    void display()
    {
        cout << "Roll No: " << Rollno<< " "<< endl;
        cout<< "Name: " << Name <<" "<<endl ;
        cout << "Marks: " << Marks << " "<<endl;
        cout<<"==================================="<<endl;
    }

    void readfile()
    {
        ifstream file("Marks.txt");

        while (file >> Rollno >> Name >> Marks) {
            if (Marks < 0 || Marks > 100) {
                throw string("Invalid marks found in file!");
            }
            display();
        }



        file.close();
    }

    void writefile()
    {
        ofstream out("Marks.txt", ios::app);
        out << Rollno << " " << Name << " " << Marks << endl;
        out.close();
    }
};
int main()
{
    try
    {
        Students s1;
        s1.getinput();
        s1.writefile();
```

```
        s1.readfile();
        cout<<"New record saved!!"<<endl;
    }
 catch(string x)
 {
    cout<<x<<endl;
 }
 return 0;
}
```

❖ Output

```
"C:\Users\beena\OneDrive\D

Enter Student Details:
Student Roll No:
8
Student Name:
Alex
Marks obtained:
88
Roll No: 2
Name: Nia
Marks: 0
================================
Roll No: 9
Name: Bina
Marks: 80
================================
Roll No: 3
Name: lina
Marks: 90
================================
Roll No: 5
Name: Bishwas
Marks: 73
================================
Roll No: 8
Name: Alex
Marks: 88
================================
New record saved!!
```


Vehicles.txt
File
Edit
View
2 Nia 0
9 Bina 80
3 lina 90
5 Bishwas 73
8 Alex 88